

Complete AI Prompt for Building Local 39 Apprenticeship System

Copy and paste this prompt to an AI coding assistant:

MASTER PROMPT: Build Local 39 Stationary Engineers Apprenticeship Management System

I need you to build a complete web-based apprenticeship management system for Local 39 Stationary Engineers. This will replace a fully paper-based process that handles up to 3,000 applicants every 2 years.

TECH STACK REQUIREMENTS

- **Backend:** Laravel 10+ (PHP 8.1+)
- **Frontend:** Vue.js 3 with Composition API
- **Database:** MariaDB/MySQL
- **Hosting:** Deployable on Replit, with production scalability
- **Authentication:** Laravel Sanctum for SPA authentication
- **File Storage:** Laravel Storage with S3-compatible backup
- **Queue System:** Laravel Queue for async tasks (emails, notifications)
- **Styling:** Tailwind CSS
- **Icons:** Heroicons or similar

SYSTEM OVERVIEW

The system must handle 4 complete modules in a sequential workflow:

MODULE 1: PRE-REGISTRATION (Public Portal)

Features:

1. **Landing Page** (before applications open):
 - Display program information (static content manageable by admin)
 - Show countdown timer to application opening date/time
 - Call-to-action: "Pre-Register to Get Notified"
2. **Pre-Registration Form:**
 - Fields: First Name, Last Name, Email, Phone Number

- Optional: Upload resume or documents (not required, for future reference)
- Validation: Email uniqueness, phone format
- reCAPTCHA v3 for bot prevention
- 3. **Confirmation System:**
 - Immediate on-screen confirmation with reference number
 - Automated email confirmation
 - 48-hour email reminder before applications open
 - Email alert when applications officially open
- 4. **Admin Controls:**
 - Set application open date/time
 - Edit landing page content
 - View pre-registration statistics
 - Export pre-registrant list to CSV/Excel

Database Tables:

pre_registrations:

- id
 - first_name
 - last_name
 - email (unique, indexed)
 - phone
 - document_path (nullable)
 - reference_number (unique)
 - notified_at (timestamp when opening alert sent)
 - created_at
 - updated_at
-

MODULE 2: APPLICATION MODULE

Features:

1. **Application Opening System:**
 - Applications open at EXACT date/time set by admin
 - Before opening: show countdown, prevent form access
 - At opening: unlock application form immediately
 - Rate limiting to prevent server overload
2. **Login/Account Creation:**
 - If pre-registered: auto-populate email (verify with reference number)
 - If new: create account during application
 - Password requirements: min 8 chars, 1 uppercase, 1 number, 1 special
 - Email verification required

3. **Application Form (Multi-step): Step 1: Personal Information**

- Full legal name (must match ID)
- Date of birth
- SSN (last 4 digits only, encrypted)
- Current address (street, city, state, zip)
- Mailing address (if different)
- Phone (primary + alternate)
- Email (confirmed)

Step 2: Educational Background

- High school name and location
- Graduation date (or GED)
- Upload high school diploma/transcript OR GED certificate (PDF/JPG, max 5MB)
- Post-secondary education (optional)

Step 3: Work Experience

- Current employment status
- Relevant work history (if any)
- Skills assessment (self-reported)

Step 4: Identification Upload

- Upload government-issued photo ID (Driver's License, State ID, or Passport)
- Front and back (if applicable)
- Validation: file type, size, image quality check

Step 5: Review & Submit

- Display all entered information
- Checkboxes: certify information is accurate, agree to terms
- Digital signature (typed name + date)
- **CRITICAL:** Timestamp recorded at exact moment of submission

4. **Post-Submission:**

- Generate unique confirmation number (format: L39-2025-XXXXXX)
- Display on-screen confirmation with:
 - Confirmation number
 - Application timestamp
 - Instructions for in-person validation
 - Validation location(s), dates, times
- Send confirmation email with same information
- Send SMS confirmation (optional)

5. **In-Person Validation (Admin Interface):**

- **Check-in System:**
 - Applicant provides confirmation number or email

- Admin pulls up application
 - Verify uploaded documents match physical documents
 - Check ID authenticity
 - **Biometric Capture:**
 - Take photo with webcam/camera (facial recognition ready)
 - OR capture fingerprint (if hardware available)
 - Store biometric data encrypted
 - **Validation Completion:**
 - Mark applicant as "Validated"
 - Record validation timestamp
 - Applicant status → "Eligible for Exam"
 - Send email/SMS: "Validation complete, exam scheduling opens soon"
6. **Admin Management:**
- View all applications (filterable, sortable, searchable)
 - Export applications to Excel
 - Bulk validation mode
 - Mark applications incomplete/flagged for review
 - Send mass emails to applicant groups

Database Tables:

applicants:

- id
- user_id (FK to users table)
- confirmation_number (unique, indexed)
- first_name, middle_name, last_name
- dob
- ssn_last4 (encrypted)
- address_street, address_city, address_state, address_zip
- mailing_address (JSON, nullable)
- phone_primary, phone_alternate
- email
- high_school_name, hs_graduation_date
- diploma_path (file path)
- id_document_front_path, id_document_back_path
- work_experience (JSON)
- application_timestamp (indexed - CRITICAL for ranking)
- validation_timestamp (indexed - CRITICAL for ranking)
- is_validated (boolean, indexed)
- validation_admin_id (FK to users)
- photo_biometric_path

- fingerprint_hash (encrypted)
- status (enum: pending, validated, exam_scheduled, exam_completed, ranked, dispatched)
- created_at, updated_at, soft_deletes

users:

- id
 - email (unique)
 - password (hashed)
 - role (enum: applicant, admin, chief, super_admin)
 - last_login_at
 - created_at, updated_at
-

MODULE 3: EXAM MANAGEMENT MODULE

Features:

- 1. Exam Scheduling (Admin):**
 - **Session Configuration:**
 - Define exam dates (e.g., 3 days)
 - Create 3 sessions per day:
 - Morning: 8:00 AM - 10:00 AM
 - Afternoon: 12:00 PM - 2:00 PM
 - Evening: 4:00 PM - 6:00 PM
 - Set capacity per session (e.g., 50 applicants)
 - Assign location/room to each session
 - **Applicant Assignment:**
 - Auto-assign validated applicants to sessions
 - Or allow applicants to self-select (first-come-first-served)
 - Send exam appointment confirmation (email/SMS)
 - Include: date, time, location, what to bring
- 2. Exam Day Check-In:**
 - **Identity Verification:**
 - Applicant enters confirmation number
 - System displays stored photo
 - Admin/proctor confirms identity match
 - OR use facial recognition if available
 - **Session Access:**
 - Once checked in, applicant receives exam portal access
 - One-time access code or auto-redirect
 - Exam locked until session start time
- 3. Exam Portal (Applicant Interface):**
 - **Pre-Exam Screen:**

- Display instructions
- List 5 sections with time limits
- "Start Exam" button (disabled until start time)
- **Exam Structure** (5 timed sections, taken sequentially): **Section 1: Reading Comprehension**
 - 25 questions
 - 25 minutes
 - Format: passages with multiple choice questions

Section 2: Math Computation

- 15 questions
- 9 minutes
- Format: basic arithmetic, no calculator

Section 3: Applied Math

- 25 questions
- 25 minutes
- Format: word problems, measurement, algebra

Section 4: Language

- 25 questions
- 18 minutes
- Format: grammar, punctuation, spelling

Section 5: Mechanical Aptitude

- 36 questions
- 20 minutes
- Format: mechanical reasoning, spatial awareness
- **Exam Functionality:**
 - Timer displayed prominently for current section
 - Questions displayed one at a time OR all at once (configurable)
 - Mark for review feature
 - Progress indicator (questions answered vs remaining)
 - **Auto-submit when time expires** (no grace period)
 - Cannot go back to previous sections
 - Warning at 5 minutes, 1 minute remaining
- **Question Delivery:**
 - Questions pulled randomly from large question bank
 - Each applicant gets different questions (from same difficulty pool)
 - No two applicants have identical exams
 - Images supported for aptitude questions

4. Post-Exam:

- Immediate auto-scoring
- Display completion message (no score revealed)
- Lock account from retaking
- Send confirmation email: "Exam completed, results under review"
- 5. **Admin Exam Management:**
 - **Question Bank Management:**
 - Add/edit/delete questions
 - Categorize by section and difficulty
 - Tag questions with skills tested
 - Import questions from Excel/CSV
 - **Session Monitoring:**
 - Live dashboard: who's checked in, in progress, completed
 - Flag suspicious activity (too fast, tab switching)
 - Ability to void/reset an exam if needed
 - **Results & Scoring:**
 - View individual exam results
 - See time spent per section
 - Export all scores to Excel
 - Statistical analysis (average, median, distribution)

Database Tables:

exam_sessions:

- id
- date
- time_slot (enum: morning, afternoon, evening)
- start_time, end_time
- location
- capacity
- filled_count
- status (enum: scheduled, in_progress, completed)
- created_at, updated_at

exam_assignments:

- id
- applicant_id (FK)
- exam_session_id (FK)
- checked_in_at (timestamp)
- check_in_verified_by (admin user_id)
- exam_started_at
- exam_completed_at

- status (enum: assigned, checked_in, in_progress, completed, voided)

exam_questions:

- id
- section (enum: reading, math_computation, applied_math, language, aptitude)
- question_text
- question_image_path (nullable)
- option_a, option_b, option_c, option_d
- correct_answer (enum: a, b, c, d)
- difficulty_level (enum: easy, medium, hard)
- tags (JSON)
- times_used (integer)
- created_at, updated_at

exam_attempts:

- id
- applicant_id (FK, unique - only one attempt)
- exam_session_id (FK)
- started_at
- completed_at
- total_score (decimal)
- section_1_score, section_2_score, section_3_score, section_4_score, section_5_score
- section_1_time, section_2_time, section_3_time, section_4_time, section_5_time
- flagged (boolean - for review)
- created_at, updated_at

exam_responses:

- id
 - exam_attempt_id (FK)
 - question_id (FK)
 - selected_answer
 - is_correct (boolean)
 - time_spent (seconds)
 - marked_for_review (boolean)
 - answered_at (timestamp)
-

MODULE 4: DISPATCH & INDENTURE MODULE

Features:

1. **Ranking System** (Automated):
 - **Ranking Criteria** (in order of priority):
 1. **Internal Promotion Path** (if applicable):
 - Must be employed by a union contractor for 6-12 months
 - Must have passing exam score ($\geq 70\%$)
 - These applicants get top priority
 2. **Exam Score** (primary factor)
 - Higher score = higher rank
 3. **Application Timestamp** (tiebreaker #1)
 - Earlier application = higher rank
 4. **Validation Timestamp** (tiebreaker #2)
 - Earlier validation = higher rank
 - **Ranking Process:**
 - Automatically triggered when all exams completed
 - Can be manually triggered by admin
 - Generates ranked list (1, 2, 3, ... N)
 - Stores rank in applicant record
 - Sends email to all: "You are ranked #X out of Y applicants"
2. **Chief Dispatch Request Portal:**
 - **User Role: Chief** (contractor representative)
 - Register/login to system
 - Submit dispatch requests
 - **Dispatch Request Form:**
 - Company name
 - Job location (city, address)
 - Job type (enum: HVAC, boiler, general maintenance, etc.)
 - Start date
 - Number of apprentices needed (usually 1)
 - Job description
 - Special requirements (if any)
 - **Submit Request:**
 - System validates request
 - Admin receives notification
 - Request enters queue
3. **Dispatch Matching System** (Admin)**:
 - **View Dispatch Requests:**
 - List all pending requests
 - Filter by location, job type, date
 - **Match Candidates:**
 - Click "Find Candidates" for a request
 - System automatically filters top 3 eligible candidates:
 - By location preference (if collected)

- By job type interest (if collected)
 - By rank (highest first)
 - Must have status: "ranked" and not already dispatched
 - Display top 3 candidates with:
 - Rank
 - Name
 - Exam score
 - Contact info
 - **Send Dispatch Offer:**
 - Select candidate(s) to notify
 - System sends email/SMS: "You have a dispatch offer"
 - Email includes:
 - Job details
 - Company name
 - Location
 - Start date
 - "Accept" or "Decline" buttons (links to portal)
 - **Response Window:** Candidate has 3 days (72 hours) to respond
- 4. **Candidate Response:**
 - **Accept Offer:**
 - Candidate logs in, clicks "Accept"
 - Status → "dispatch_accepted"
 - Admin receives notification
 - System removes candidate from available pool
 - **Decline Offer:**
 - Candidate clicks "Decline"
 - Must provide reason (dropdown: not ready, location too far, other)
 - Rank position maintained
 - Available for future dispatches
 - **No Response (fallback):**
 - After 72 hours, system auto-notifies next candidate
 - Original candidate marked "non-responsive" for this dispatch
 - Admin can manually override
- 5. **Indenture Agreement (Digital):**
 - **Generate Indenture Form:**
 - Once candidate accepts, admin clicks "Generate Indenture"
 - System auto-fills PDF template with:
 - Apprentice name
 - SSN
 - Company name
 - Start date
 - Term length (typically 4 years)
 - Wage schedule
 - Training requirements
 - **E-Signature Workflow:**
 - Send to apprentice for signature (via DocuSign API or similar)

- Send to employer for signature
- Send to union representative for signature
- Once all signed, system stores final PDF
- **Post-Signature:**
 - Apprentice status → "indentured"
 - Send welcome email with:
 - Training schedule
 - Union hall location
 - Contact information
 - What to expect on first day
 - **Admin Manual Step** (for now):
 - Admin manually enters apprentice into UnionNet system
 - Checkbox in system: "Entered into UnionNet"
 - Future: API integration to automate
- 6. **Dispatch Dashboard (Admin):**
 - View all dispatches (past, pending, completed)
 - Track response rates
 - See which candidates are in indenture process
 - Export dispatch reports

Database Tables:

rankings:

- id
- applicant_id (FK, unique)
- rank (integer, indexed)
- exam_score
- application_timestamp
- validation_timestamp
- is_internal_promotion (boolean)
- employment_start_date (nullable)
- ranked_at (timestamp)
- created_at, updated_at

dispatch_requests:

- id
- chief_user_id (FK)
- company_name
- job_location_address
- job_location_city
- job_location_state

- job_type
- start_date
- positions_needed (default: 1)
- job_description (text)
- special_requirements (text, nullable)
- status (enum: pending, in_progress, filled, cancelled)
- created_at, updated_at

dispatch_offers:

- id
- dispatch_request_id (FK)
- applicant_id (FK)
- offered_at (timestamp)
- response_deadline (timestamp)
- response_at (timestamp, nullable)
- response_status (enum: pending, accepted, declined, expired)
- decline_reason (text, nullable)
- created_at, updated_at

indentures:

- id
- dispatch_offer_id (FK)
- applicant_id (FK)
- company_name
- start_date
- term_length_years (default: 4)
- wage_schedule (JSON)
- document_template_path
- signed_document_path (nullable)
- apprentice_signed_at (timestamp, nullable)
- employer_signed_at (timestamp, nullable)
- union_signed_at (timestamp, nullable)
- status (enum: draft, pending_signatures, fully_executed)
- entered_in_unionnet (boolean, default: false)
- unionnet_id (nullable)
- created_at, updated_at

CROSS-CUTTING REQUIREMENTS

1. Role-Based Access Control (RBAC)

Roles:

- **Public:** Can pre-register, view landing page
- **Applicant:** Can apply, take exam, view dispatch offers, sign indenture
- **Admin:** Can manage all modules, validate applications, grade exams, process dispatches
- **Chief:** Can submit dispatch requests, view their requests
- **Super Admin:** Full system access, user management, settings

Permissions (use Spatie Laravel-Permission):

- view_applications
- validate_applications
- manage_exams
- view_exam_results
- manage_dispatch_requests
- send_dispatch_offers
- generate_indentures
- manage_users
- view_audit_logs
- edit_system_settings

2. Security Requirements

- **Authentication:**
 - Laravel Sanctum for API authentication
 - 2FA optional (via email or SMS)
 - Rate limiting on login attempts
 - Password reset via email
- **Data Encryption:**
 - Encrypt SSN, biometric data at rest
 - Use Laravel's encryption helpers
 - HTTPS only in production
- **File Upload Security:**
 - Validate file types (whitelist: PDF, JPG, PNG)
 - Scan for malware (use ClamAV or similar)
 - Store files outside public directory
 - Generate unique filenames

- Max file size: 5MB per file
- **Exam Security:**
 - Prevent tab switching (log attempts)
 - Disable right-click, copy-paste in exam portal
 - Randomize question order
 - One exam attempt per applicant (strictly enforced)

3. Audit Trail

Log all critical actions:

- User logins/logouts
- Application submissions
- Application edits by admin
- Validation actions
- Exam starts/completions
- Exam score changes
- Dispatch offers sent
- Dispatch responses
- Indenture generations
- Any data deletions/modifications

Audit Log Structure:

audit_logs:

- id
- user_id (who performed action)
- auditable_type (model name)
- auditable_id (record ID)
- action (enum: created, updated, deleted, viewed, etc.)
- old_values (JSON)
- new_values (JSON)
- ip_address
- user_agent
- created_at

Use **Laravel Auditing** package for automatic logging.

4. Notifications

Email Notifications:

- Pre-registration confirmation

- Application opening alert (48 hours before)
- Application opening alert (at open time)
- Application submission confirmation
- Validation completion
- Exam appointment reminder (24 hours before)
- Exam completion confirmation
- Ranking notification
- Dispatch offer
- Dispatch offer reminder (48 hours before deadline)
- Indenture signature request
- Indenture fully executed confirmation

SMS Notifications (optional, Phase 2):

- Same as email, but shorter messages
- Use Twilio or similar

In-App Notifications:

- Dashboard alerts for applicants
- Real-time notifications for admins

5. Reporting & Analytics

Admin Dashboard Widgets:

- Total pre-registrations
- Total applications submitted
- Total validated applicants
- Total exams completed
- Average exam score
- Dispatches pending/completed
- Indentures pending/completed

Exportable Reports:

- All applications (Excel/CSV)
- Exam results (Excel/CSV)
- Rankings list (Excel/PDF)
- Dispatch history (Excel/CSV)
- Audit log (Excel/CSV)

Charts/Graphs:

- Applications over time (line chart)
- Exam scores distribution (histogram)

- Dispatch success rate (pie chart)
- Top performing locations (bar chart)

6. Multi-Language Support (Phase 2+)

- Use Laravel localization
- Support English (default) and Spanish
- Translatable: UI text, emails, instructions
- Language switcher in navigation

7. System Settings (Admin Panel)

Configurable Settings:

- Application open date/time
- Application close date/time (optional)
- Exam session dates/times
- Passing exam score threshold (default: 70%)
- Dispatch response deadline (default: 72 hours)
- Email templates (customizable)
- Landing page content
- Program information text
- Contact information

8. Performance & Scalability

- **Caching:**
 - Cache landing page content
 - Cache ranked list
 - Use Redis for session storage
 - **Queue Jobs:**
 - Email sending
 - SMS sending
 - Ranking calculation
 - PDF generation
 - Large exports
 - **Database Optimization:**
 - Index frequently queried columns
 - Use eager loading to prevent N+1 queries
 - Paginate large result sets
 - **File Storage:**
 - Use cloud storage (S3 or compatible)
 - CDN for static assets
 - Automated backups
-

DELIVERABLES

Please provide the following:

Phase 1: Foundation (Week 1-2)

1. Complete Laravel project structure
2. Database schema with all migrations
3. Seeders for roles, permissions, sample data
4. Authentication system (login, register, password reset)
5. Admin dashboard skeleton
6. Basic routing (API + web)

Phase 2: Module 1 - Pre-Registration (Week 3)

7. Landing page (Vue component)
8. Countdown timer
9. Pre-registration form
10. Email confirmation system
11. Admin: view pre-registrants, set open date

Phase 3: Module 2 - Application (Week 4-5)

12. Application form (multi-step, Vue)
13. File upload functionality
14. Application timestamp recording
15. Confirmation number generation
16. In-person validation interface (admin)
17. Biometric capture (photo storage)
18. Application management dashboard (admin)

Phase 4: Module 3 - Exam (Week 6-7)

19. Exam session scheduling (admin)
20. Question bank management (admin)
21. Check-in system (admin)
22. Exam portal (applicant, Vue)
23. Timed sections with auto-submit
24. Auto-scoring system
25. Exam monitoring dashboard (admin)

Phase 5: Module 4 - Dispatch (Week 8-9)

26. Ranking algorithm
27. Ranking notification emails

28. Chief portal (dispatch request form)
29. Dispatch matching system (admin)
30. Candidate notification system
31. Response tracking (accept/decline)
32. Indenture PDF generation
33. E-signature workflow (basic)

Phase 6: Cross-Cutting Features (Week 10)

34. Complete RBAC implementation
35. Audit logging throughout
36. All email/notification templates
37. Reporting dashboard
38. Export functionality (Excel/PDF)
39. System settings panel

Phase 7: Testing & Documentation (Week 11-12)

40. Unit tests for critical functions
41. Feature tests for workflows
42. API documentation (Swagger)
43. User manuals (Admin, Applicant, Chief)
44. Deployment guide for Replit
45. Database backup/restore procedures

DESIGN REQUIREMENTS

UI/UX Guidelines:

- **Mobile-responsive** (Bootstrap 5 or Tailwind CSS)
- **Accessible** (WCAG 2.1 AA compliant)
- **Color scheme:** Union/professional (blues, grays, white)
- **Typography:** Clear, readable (minimum 16px base)
- **Forms:** Clear labels, inline validation, helpful error messages
- **Loading states:** Show spinners during async operations
- **Success/error feedback:** Toast notifications or alerts

Key User Flows:

1. Pre-registration → Email confirmation → Reminder → Application
2. Application → Validation → Exam → Ranking → Dispatch
3. Chief: Request → Admin review → Candidate matching → Offer → Indenture
4. Admin: Manage all modules, view analytics, troubleshoot

TECHNICAL SPECIFICATIONS

API Endpoints (RESTful):

Public/Guest:

- GET /api/countdown - Get application open time
- POST /api/pre-register - Submit pre-registration

Applicant (Auth required):

- POST /api/application - Submit application
- POST /api/application/upload - Upload documents
- GET /api/exam/session - Get assigned exam session
- POST /api/exam/start - Start exam
- POST /api/exam/answer - Submit answer
- POST /api/exam/complete - Complete exam section
- GET /api/ranking - Get own ranking
- GET /api/dispatch-offers - Get dispatch offers
- POST /api/dispatch-offers/{id}/respond - Accept/decline offer

Chief (Auth required):

- POST /api/dispatch-requests - Submit dispatch request
- GET /api/dispatch-requests - View own requests

Admin (Auth required):

- GET /api/admin/applications - List all applications
- POST /api/admin/applications/{id}/validate - Validate application
- POST /api/admin/exam-sessions - Create exam session
- GET /api/admin/exam-results - View all exam results
- POST /api/admin/rankings/generate - Generate rankings
- GET /api/admin/dispatch-requests - View all dispatch requests
- POST /api/admin/dispatch-offers - Send dispatch offer
- POST /api/admin/indentures/generate - Generate indenture PDF

Frontend Components (Vue.js):

Public:

- LandingPage.vue
- CountdownTimer.vue

- `PreRegistrationForm.vue`

Auth:

- `Login.vue`
- `Register.vue`
- `PasswordReset.vue`

Applicant:

- `Dashboard.vue`
- `ApplicationForm.vue` (multi-step)
- `FileUpload.vue`
- `ExamPortal.vue`
- `ExamQuestion.vue`
- `ExamTimer.vue`
- `RankingView.vue`
- `DispatchOffers.vue`

Chief:

- `ChiefDashboard.vue`
- `DispatchRequestForm.vue`
- `MyRequests.vue`

Admin:

- `AdminDashboard.vue`
- `ApplicationList.vue`
- `ValidationInterface.vue`
- `ExamSessionManager.vue`
- `QuestionBankManager.vue`
- `ExamMonitor.vue`
- `RankingsManager.vue`
- `DispatchManager.vue`
- `IndentureGenerator.vue`
- `Reports.vue`
- `SystemSettings.vue`

ADDITIONAL NOTES

- Use **Laravel Sanctum** for SPA authentication
- Use **Laravel Queue** with database driver (upgrade to Redis in production)
- Use **Laravel Notifications** for email/SMS
- Use **Laravel Storage** with S3 driver (or compatible)

- Use **Spatie Laravel-Permission** for RBAC
 - Use **Laravel Auditing** for audit logs
 - Use **Maatwebsite Excel** for exports
 - Use **DomPDF** for PDF generation
 - Use **Intervention Image** for image processing
 - Write clean, documented code
 - Follow PSR-12 coding standards
 - Use meaningful variable/function names
 - Add comments for complex logic
 - Use Vue Composition API (not Options API)
 - Use Pinia for state management
 - Use Axios for API calls
 - Use Vue Router for navigation
-

START HERE

Begin by:

1. Setting up the Laravel project structure
2. Creating all database migrations
3. Implementing authentication with Sanctum
4. Building the admin dashboard skeleton
5. Then proceed module by module (1 → 2 → 3 → 4)

Ask me questions if anything is unclear. Provide code for each component as you build it.

Ready to start? Please confirm you understand the requirements and begin with Phase 1.

END OF PROMPT



How to Use This Prompt

1. **Copy the entire prompt above**
2. **Paste it into:**
 - Claude (Anthropic) - Best for complex coding
 - ChatGPT-4 (OpenAI) - Good for full-stack
 - Replit AI - If building directly in Replit
 - Cursor AI - For VS Code integration
 - Any AI coding assistant

3. **Follow up with:**

- "Start with Phase 1"
- "Show me the database migrations first"
- "Build Module 1 completely"
- "I need the Vue components for the exam portal"

4. **Iterative approach:**

- Ask for one module at a time
- Review and test each module
- Request modifications as needed
- Move to next module when satisfied

Would you like me to help you get started with any specific part of the system?